

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	D. Yaswanth Sri Dana Ganesh	Reg. No.:	192425351
Date:		Duration:	60 minutes

Code of Conduct

I D. Yaswanth Sri Dana Ganesh (192425351) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dana Ganesh

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. Introduction

This document designs a complete CI/CD (Continuous Integration and Continuous Deployment) pipeline for the Satellite Data Visualization Platform, automating the flow from a developer's code commit through build, test, quality and security checks, packaging, and deployment into development, staging, and production environments. It builds directly on the functional and non-functional requirements (FR1–FR8, NFR1–NFR8) and the 27-case test suite already defined for this platform, so that automated testing in the pipeline reuses that same suite rather than duplicating effort.

The platform's nature makes a disciplined pipeline especially important: it ingests live data from external NASA/NOAA APIs that change independently of the codebase, it is relied upon by disaster-management agencies during time-critical events (making uptime and safe rollback essential), and it exposes an authenticated export feature that must be protected by security scanning at every release. The pipeline design below addresses each of these concerns explicitly.

2. CI/CD Requirement Analysis

Before designing the pipeline stages, the specific CI/CD needs of this project were identified from its problem statement and prior requirement analysis. These needs (CR1–CR8) drive every tool and stage decision made later in this document.

Req. ID	CI/CD Requirement
CR1	Automated build for both the backend (data-ingestion / processing services) and the frontend (map and chart visualization UI) on every commit.
CR2	Automated unit, integration, and functional regression testing on every pull request, reusing the test suite defined in the project's Test Case Design document.
CR3	Static code analysis and dependency / container vulnerability scanning before code can be merged or deployed.
CR4	Reproducible, containerized build artifacts to guarantee environment parity across development, staging, and production.
CR5	Automatic deployment to a staging environment for QA and stakeholder validation after every successful merge to the main branch.
CR6	A manual approval gate and automated smoke tests before any release reaches production, given the platform's use by disaster-management agencies.
CR7	A fast, reliable rollback mechanism to revert a faulty production release with minimal downtime.
CR8	Automated notifications to the development team on build, test, and deployment failures.

3. Pipeline Architecture & Workflow

The pipeline is organized into nine sequential stages, moving from source control through to production monitoring, with an automated rollback path guarding the final release step. Each stage has a single clear responsibility and a defined trigger, so failures are caught as early as possible — a principle often called “shift-left” testing.

Pipeline Stage	Purpose	Trigger
1. Source Code Management	Developers commit to feature branches and open pull requests against main.	On every git push
2. Build	Backend (Python data-processing services) and frontend (React/Leaflet map UI) are installed, compiled, and containerized.	On PR open/update & merge to main
3. Automated Testing	Unit, integration, and functional regression tests run against the build.	Immediately after build succeeds
4. Code Quality & Security Analysis	Static analysis, linting, and dependency/container vulnerability scans run; a quality gate is enforced.	After tests pass
5. Packaging	A versioned Docker image is built and pushed to the container registry.	After the quality gate passes
6. Deploy to Staging	The new image is automatically deployed to staging; smoke tests run.	On merge to main
7. Manual Approval Gate	A release manager reviews staging results and approves promotion to production.	After staging smoke tests pass
8. Deploy to Production	A canary / blue-green deployment gradually shifts live traffic to the new version.	After manual approval
9. Monitoring & Rollback	Health checks, error rates, and latency are watched continuously; an automatic rollback triggers if thresholds are breached.	Continuous, post-deployment

CI/CD Pipeline Workflow — Satellite Data Visualization Platform

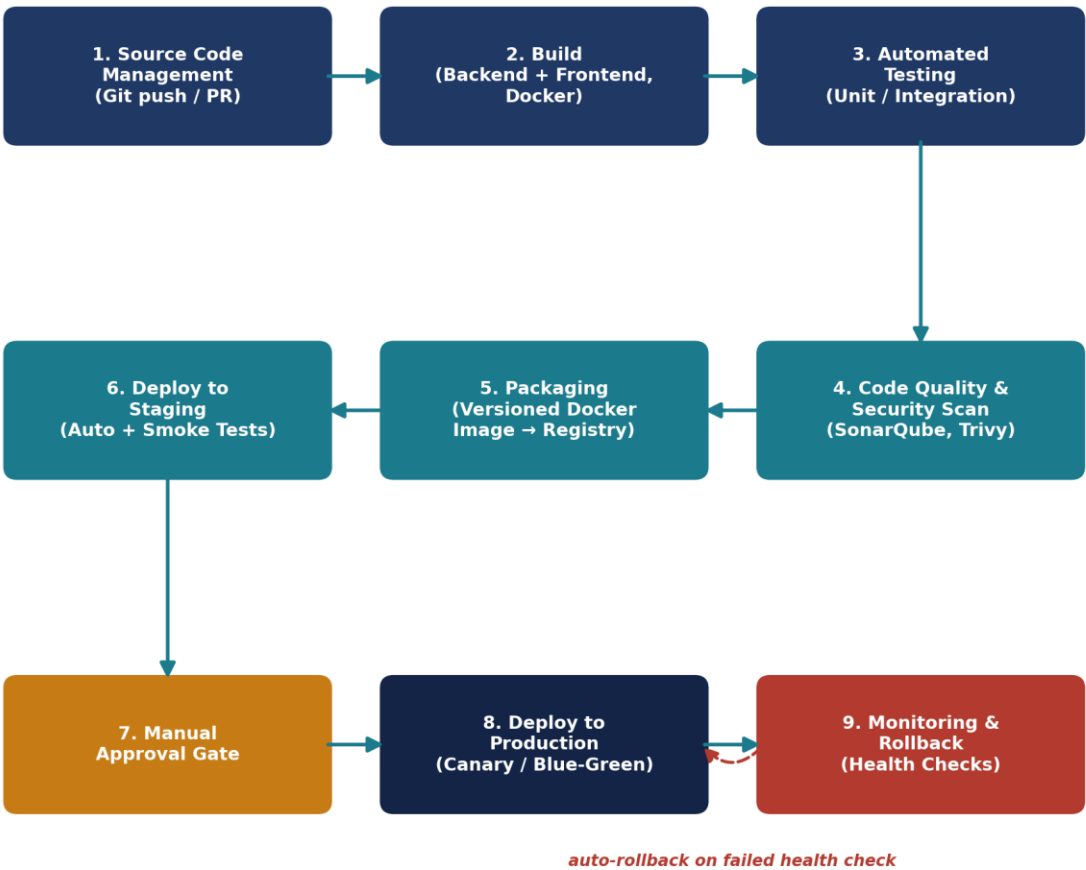


Figure 1: CI/CD Pipeline Workflow — Satellite Data Visualization Platform

3.2 Sample Pipeline Configuration

The excerpt below illustrates how the first four pipeline stages are expressed as pipeline-as-code (GitHub Actions). The full workflow additionally defines the staging/production deployment jobs described in Sections 3 and 6, omitted here for brevity.

```
name: CI Pipeline

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Build backend image
        run: docker build -t satviz-backend:${{ github.sha }} ./backend

      - name: Build frontend
        run: npm ci && npm run build --prefix frontend

  test:
    needs: build
    runs-on: ubuntu-latest

    steps:
      - name: Run unit & integration tests
```

```
run: pytest --cov=app --cov-fail-under=80
```

```
- name: Run functional regression suite
```

```
run: npx cypress run --spec "cypress/e2e/regression/*.cy.js"
```

quality-and-security:

```
needs: test
```

```
runs-on: ubuntu-latest
```

```
steps:
```

```
- name: SonarQube scan
```

```
uses: sonarsource/sonarqube-scan-action@v2
```

```
- name: Container vulnerability scan
```

```
run: trivy image satviz-backend:${{ github.sha }}
```

package:

```
needs: quality-and-security
```

```
runs-on: ubuntu-latest
```

```
steps:
```

```
- name: Push versioned image
```

```
run: docker push ghcr.io/org/satviz-backend:${{ github.sha }}
```

4. Tools & Technology Selection

The following tools were selected for each stage, chosen for their strong fit with a containerized, Python/JavaScript-based data-visualization platform and for their ability to enforce automated quality and security gates.

Pipeline Stage	Tool(s)	Justification
Source Code Management	GitHub (Git)	Industry-standard version control with native pull-request review and branch-protection rules that block merges until

Pipeline Stage	Tool(s)	Justification
		CI passes.
CI/CD Orchestration	GitHub Actions	YAML-based pipelines-as-code that live alongside the repository, with a large marketplace of pre-built actions for testing, scanning, and deployment.
Build & Containerization	Docker, npm / pip	Packages the Python data-processing backend and the React/Leaflet frontend into consistent, portable images, eliminating “works on my machine” issues.
Automated Testing	PyTest, Cypress, Postman/Newman	PyTest covers backend unit/integration logic; Cypress automates key UI journeys drawn from the existing 27-case test suite; Newman runs API contract tests.
Code Quality Analysis	SonarQube, ESLint / Pylint	Flags code smells, duplication, and complexity, and enforces a minimum test-coverage threshold as a merge gate.
Security Scanning	Trivy (container images), OWASP Dependency-Check	Detects known CVEs in both the built Docker images and third-party library dependencies before they reach staging.
Artifact Registry	GitHub Container Registry	Stores versioned, immutable image tags so any past release can be redeployed exactly — essential for rollback.
Infrastructure as Code	Terraform	Defines staging and production infrastructure declaratively, so environments stay reproducible and auditable.
Deployment Orchestration	Kubernetes + Helm	Enables canary/blue-green rollouts, auto-scaling under concurrent load (NFR4), and self-healing of failed pods.
Monitoring & Alerting	Prometheus + Grafana, Sentry	Tracks latency and error-rate metrics (NFR1) against dashboards and thresholds, and captures application-level exceptions.
Notifications	Slack / Microsoft Teams webhooks	Sends immediate alerts to the development channel on build, test, or deployment failures, and on every rollback event.

5. Automated Testing Strategy

Testing is layered so that cheap, fast checks run first and catch most defects before the slower, broader checks run. The functional/UI regression layer directly automates representative journeys from the project's existing 27 manually designed test cases (e.g., TC-01 data ingestion, TC-12 invalid date-range handling, TC-21 unauthenticated export, TC-27 concurrency smoke test), so the two documents work together rather than duplicating effort.

Automated Testing Pyramid Integrated into the Pipeline

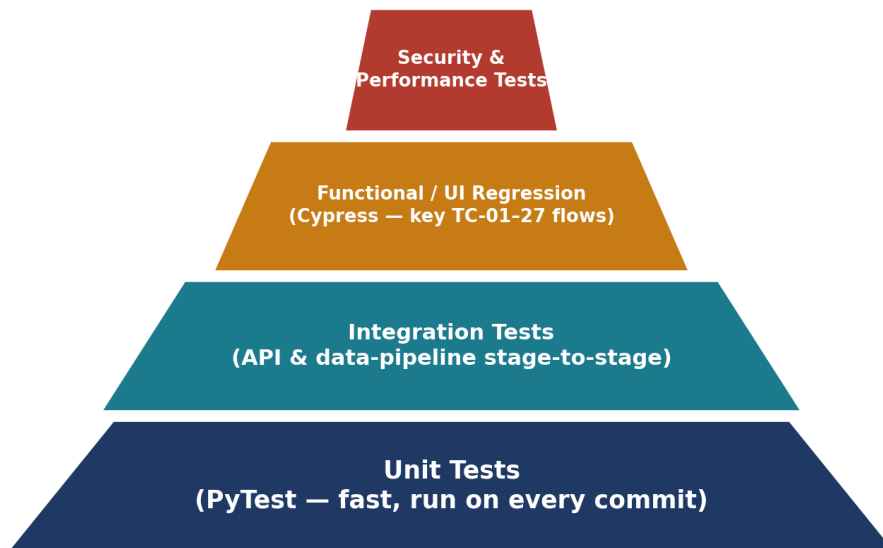


Figure 2: Automated Testing Pyramid Integrated into the Pipeline

A build is blocked from progressing to the next stage unless it passes the following quality gate:

- Unit test coverage of at least 80% on changed code (SonarQube coverage gate).
- Zero critical or high-severity vulnerabilities in dependency and container scans (Trivy / OWASP Dependency-Check).
- SonarQube quality gate passed (no new code smells above the configured severity threshold).
- All Priority-0 functional regression tests (covering FR1–FR8 core journeys) pass in the Cypress suite.

6. Deployment Strategy

Three environments form a promotion pipeline, each with a distinct purpose and audience, so that a change is validated progressively before it reaches real end users such as meteorologists and disaster-management agencies.

Environment	Trigger	Purpose	Deployment Strategy
Development	Every push to a feature branch	Fast feedback for the developer; an ephemeral preview deployment mirrors the feature in isolation.	Auto-deployed per branch; torn down when the branch is merged or closed.
Staging / UAT	Merge to main (after CI passes)	QA validation and stakeholder sign-off against production-like data and configuration before release.	Automatic full deployment, followed by automated smoke tests.
Production	Manual approval after staging sign-off	Serves real users; must minimise risk and downtime during release.	Canary release (small traffic slice first) or blue-green switch, with automatic rollback on failure.

Environment Promotion Flow with Rollback Path

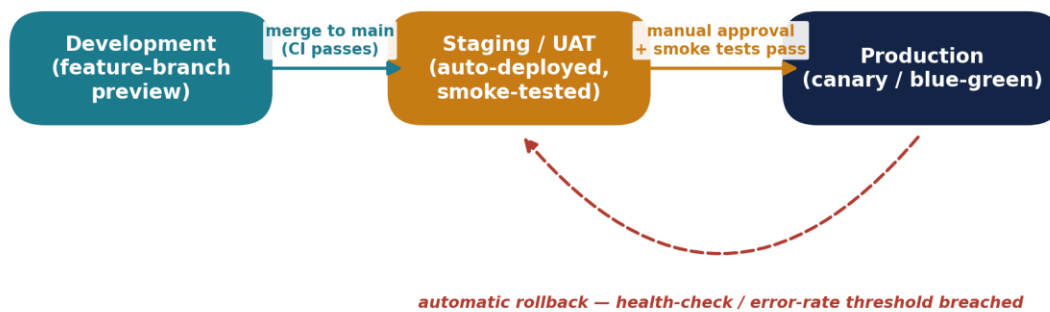


Figure 3: Environment Promotion Flow with Rollback Path

7. Security, Quality Gates, Notifications & Rollback

The pipeline enforces security and reliability at multiple points rather than relying on a single check:

- Secrets management: API keys and credentials (e.g., NASA/NOAA API keys, database credentials) are stored in a secrets vault and injected at deploy time — never committed to source control.
- Least-privilege access: the deployment pipeline's service account has only the permissions required for its target environment; production deploy credentials are separate from staging.
- Mandatory code review: branch-protection rules require at least one peer approval and a passing CI run before any merge to main.
- Automated rollback: if post-deployment health checks, error rates, or latency breach configured thresholds during the canary bake period, the pipeline automatically redeploys the last known-good image.
- Manual rollback: a release manager can trigger a one-command Helm rollback to any previous versioned image at any time, independent of the automatic trigger.
- Notifications: Slack/Teams alerts fire on build failure, test failure, failed quality/security gate, successful production release, and every rollback event, so the team is never out of the loop.

8. Ethical Considerations

The CO4 outcome for this course explicitly emphasises ethics alongside security and reliability. In this pipeline, that translates into concrete, automatable checks rather than a one-off statement of intent:

- Data licensing & attribution: NASA and NOAA impose specific reuse and attribution terms on their data. An automated check confirms source metadata and attribution survive end-to-end into exported datasets, so downstream users are never misled about data provenance.
- Transparent audit trail: every build, test result, approval, deployment, and rollback is logged immutably — who approved it, when, and against which commit — so decisions affecting a platform used in disaster response remain accountable and reviewable after the fact.
- Responsible resource usage: auto-scaling and scheduled scale-down of non-production environments

outside working hours limit unnecessary cloud compute and the platform's environmental footprint.

- Open-source licence compliance: the dependency-scanning stage also checks third-party library licences, avoiding restrictive or incompatible licences in a platform intended for broad public-sector reuse.
- Equitable access during rollouts: canary releases are monitored for accessibility regressions (NFR8) as well as errors, so a deployment cannot silently degrade usability for non-expert or assistive-technology users.

9. Pipeline Metrics & KPIs (DORA Metrics)

Pipeline health is tracked using the four DORA (DevOps Research and Assessment) metrics, the industry-standard indicators of CI/CD maturity, surfaced on the Grafana dashboard defined in Section 4.

DORA Metric	What It Measures	Target for This Platform
Deployment Frequency	How often code is successfully deployed to production	Weekly, aligned with the pace of upstream NASA/NOAA data-source changes
Lead Time for Changes	Time from commit to running in production	Under 1 day for a change that passes all quality gates
Change Failure Rate	Percentage of deployments causing an incident or rollback	Under 15%
Mean Time to Recovery (MTTR)	Time to restore service after a failed deployment	Under 15 minutes, via automatic rollback

10. Risks & Mitigation

Designing the pipeline also means anticipating how it can fail. The table below lists the most significant risks identified and how each is mitigated.

Risk	Impact	Mitigation
Flaky automated tests intermittently fail the pipeline	Blocks valid releases; erodes trust in CI results	Quarantine flaky tests into a separate non-blocking job; track and fix within a defined SLA.
Secrets accidentally committed to source control	Credential leak; potential security breach	Pre-commit secret-scanning hook plus mandatory secrets vault; immediate revoke-and-rotate if detected.
Third-party NASA/NOAA API rate limits hit during tests	Test failures unrelated to actual code quality	Use recorded/mock API fixtures for CI; reserve live-API calls for a scheduled nightly job.
Canary traffic misrouted or health checks misconfigured	A faulty release reaches more users than intended	Automatic rollback tied to independently verified health-check and error-rate thresholds; validated via periodic rollback drills.
Approval bottleneck delays urgent hotfixes	Slows critical fixes during active disaster events	Expedited hotfix path (Section 3.1) with a secondary on-call approver.

11. Conclusion

This document has designed a nine-stage CI/CD pipeline for the Satellite Data Visualization Platform, identified eight CI/CD-specific requirements (CR1–CR8), selected and justified eleven tools across source control, build, testing, quality, security, and deployment, defined a three-environment promotion strategy with both automatic and manual rollback mechanisms, and set out a trunk-based branching model with a sample pipeline-as-code configuration.

By reusing the platform's existing 27-case test suite inside the automated testing layer and gating every release on coverage, static analysis, and vulnerability scanning, the pipeline gives reasonable confidence that changes reaching production have been verified for correctness, security, and reliability — which matters directly for a platform that disaster-management agencies depend on during time-critical events. The added ethical safeguards, DORA-metric targets, and documented risk mitigations round this out into a pipeline that is not only automated but also accountable and measurably reliable, directly addressing the CO4 outcome's emphasis on security, reliability, and ethics.